Beyond the GPU

Wafer-scale computing and the ALCF AI Testbed

Ahmedur Rahman Shovon · Argonne National Laboratory

Argonne NATIONAL LABORATORY

U.S. DEPARTMENT of ENERGY

01

Announcements

Today

- 50 min: *Beyond the GPU* (this talk)
- 25 min: *Microbenchmarking NVIDIA's Blackwell Architecture: An in-depth Architectural Analysis*

Next class (Monday)

- Prof. Kumar will be back
- Volunteers needed to present *FABRICA: Agentic CUDA-to-CSL Translation and Optimization for Wafer-Scale Systems*

02

Argonne

Argonne National Laboratory

1946
the first US national laboratory

3,836
employees, 769 grad/undergrad students

4th
Aurora, 4th world fastest supercomputer

21
research divisions, 6 national user facilities

3
Nobel laureates among its researchers

250+
open-source software tools and models

anl.gov/our-history/timeline · anl.gov by the numbers FY2024 · TOP500 June 2026

Where the computing research lives

BIO
Biosciences

CPS
Computational Science

DSL
Data Science and Learning

EVS
Environmental Science

MCS
Mathematics and Computer Science
MPICH, PETSc, runtimes, workflows

ALCF
Leadership Computing Facility
Aurora · Polaris · Sophia · AI Testbed

Computing, Environment, and Life Sciences (CELS) directorate: 5 divisions and the national user facility.

Internship at Argonne

Undergraduate

SULI
Community College Internship
Lee Teng Fellowship

Graduate

Givens Associates
W. J. Cody Associates
DOE SCGSR

Seasonal

Seasonal Internship Program
Research Aide
Pre-College (ERI)

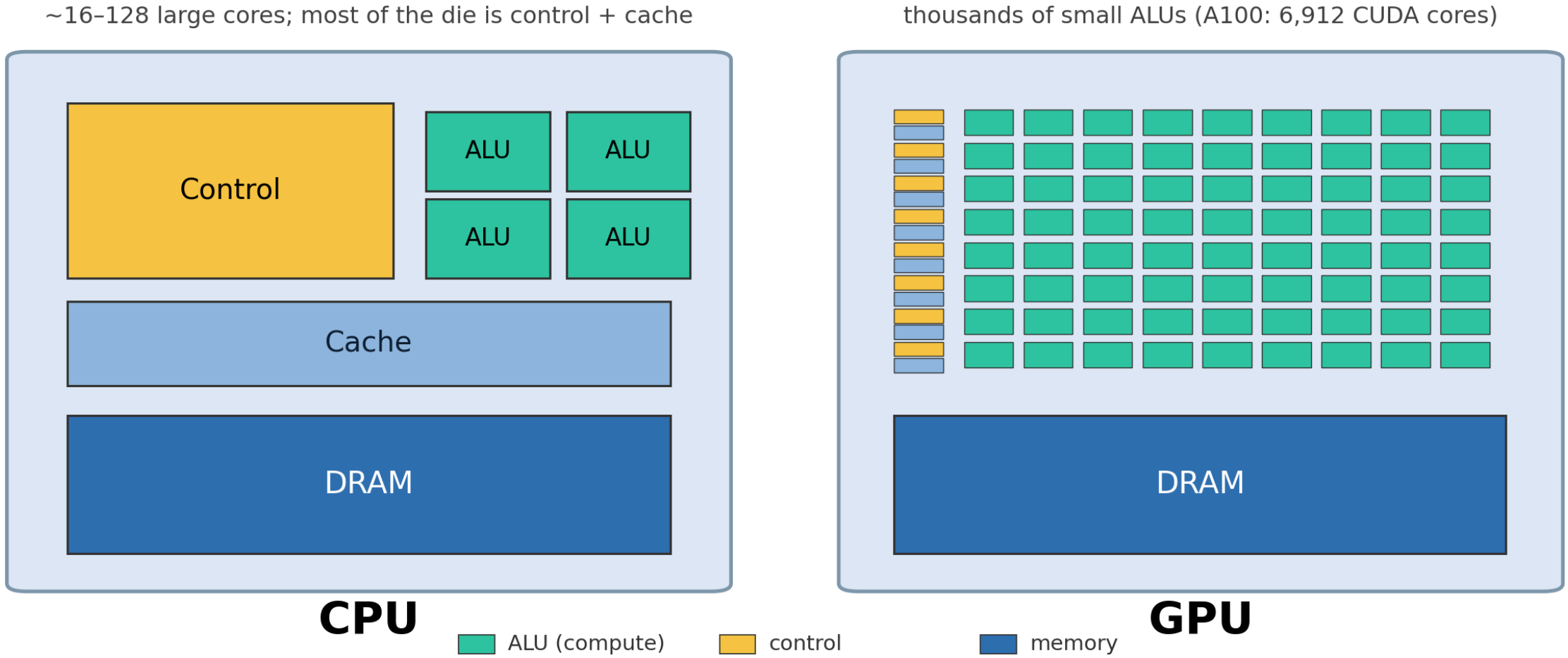
More than 1,000 interns joined Argonne this summer — across AI, quantum, materials and more.

> Undergraduate: anl.gov/education/undergraduate-internship-opportunities · Graduate: anl.gov/education/graduate-internships · Seasonal: anl.gov/education/seasonal-internship-program

03

Basics

Inside a processor: compute and memory

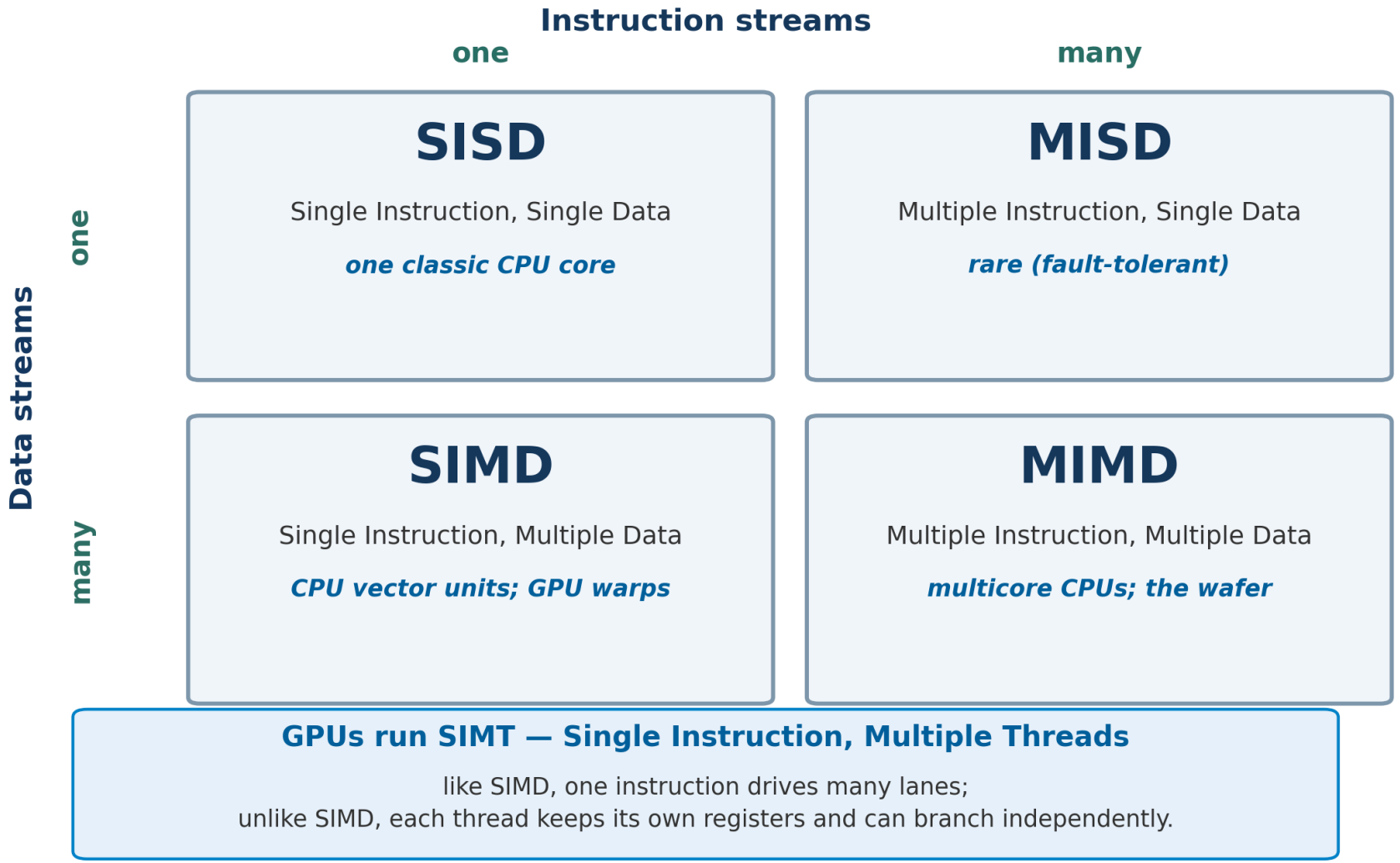


Green is compute (ALUs), blue is memory, yellow is control

A CPU spends most of its die on control and cache; a GPU spends it on ALUs

Cores: a server CPU has ~16–128; an NVIDIA A100 GPU has 6,912 CUDA cores (108 SMs × 64)

Flynn's taxonomy



Classify by instruction streams × data streams

A program is limited by data or by arithmetic

Vector add — the processor mostly waits



Matrix multiply — the processor stays busy



time →

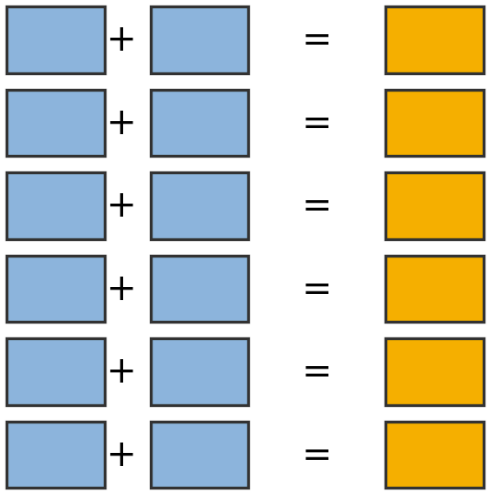
> Memory bound: waiting for data. Compute bound: doing arithmetic
>
> Arithmetic intensity, operations per byte — decides which

Three example kernels

A "kernel" is a small operation a program repeats over lots of data

1. Vector add

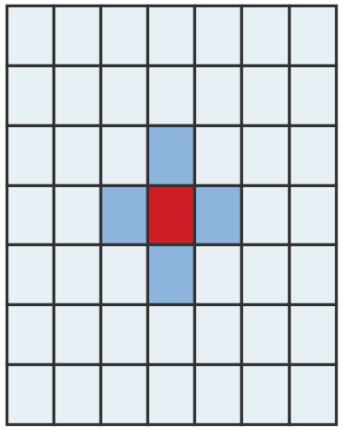
add two lists number-by-number



1 add per number:
lots of data, little math
→ memory bound

2. Stencil

update each grid cell from its neighbours

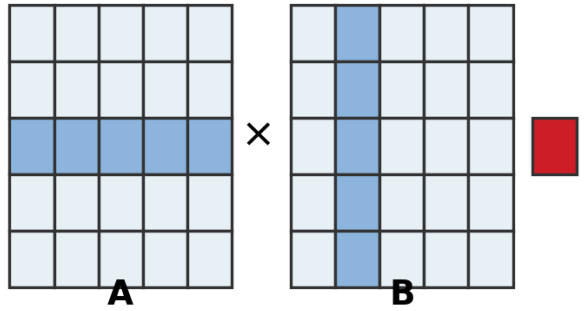


new cell (red) = itself + neighbours (blue)

one sweep of the grid = one "step"
heat, waves, weather, image blur
→ memory bound

3. Matrix multiply

rows times columns of two tables



one result cell = a row × a column

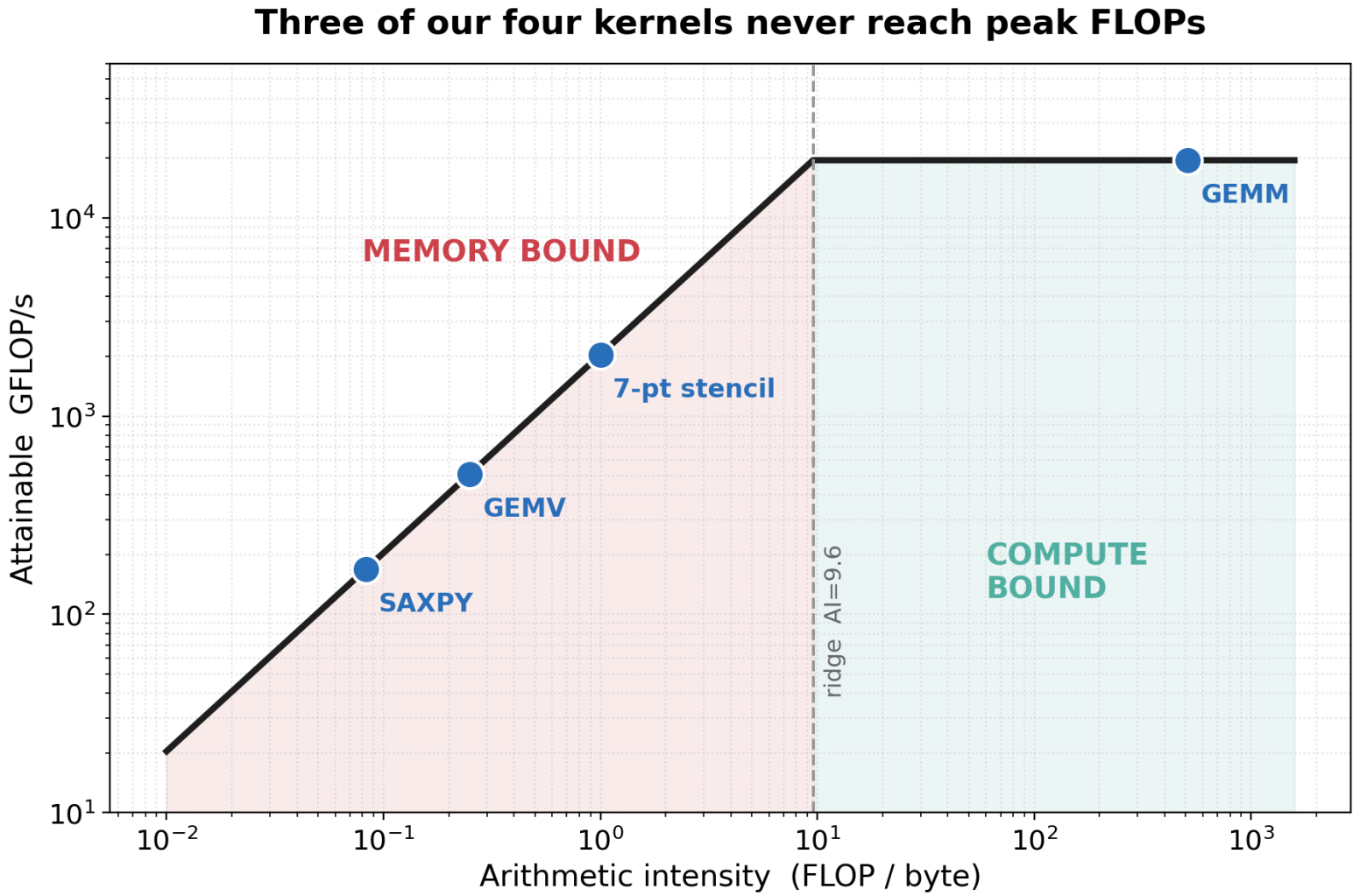
lots of math, data reused a lot
core of neural nets & linear algebra
→ compute bound

A **kernel** is the small piece of math a program runs over and over on lots of data

Vector add: add two lists. **Stencil**: update each grid cell from its neighbours (blur, heat spread, waves). **Matrix multiply**: rows × columns

Vector add and stencil are memory bound; matrix multiply is compute bound — arithmetic intensity rises left to right

The roofline: where each kernel can land



The diagonal is the memory-bandwidth limit; the flat ceiling is peak compute

Low arithmetic intensity keeps a kernel on the diagonal — memory bound

Bandwidth, drawn to scale



Drawn to scale. Bandwidth is the rate data reaches the cores.

Bandwidth is the rate at which data reaches the cores

Many scientific programs reach this limit before they run out of cores

Vendor specifications: DDR5, NVIDIA A100 HBM2

04

Machines

Aurora



10,624	63,744	21,248	20.4 PB	exascale
COMPUTE NODES	INTEL GPUS	XEON CPUS	SYSTEM MEMORY	10^{78} OPS / SECOND

ALCF Aurora · Image: Argonne National Laboratory

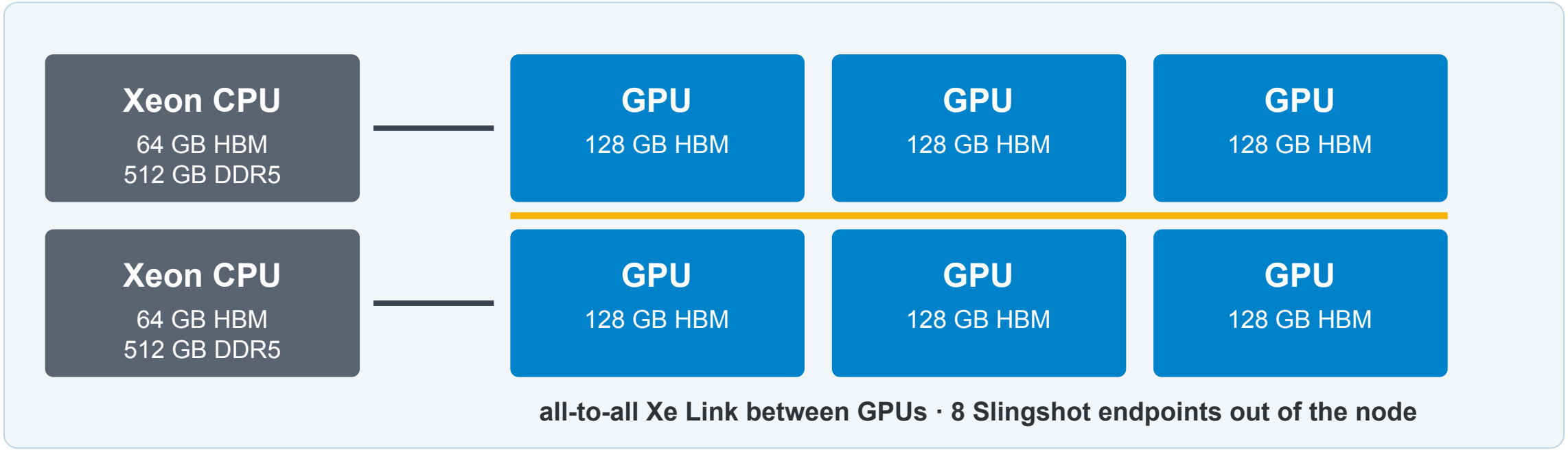
Polaris



560	2,240	44 PF	HPE
COMPUTE NODES	NVIDIA A100 GPUS	TENSOR-CORE FP64 PEAK	APOLLO 6500 GEN10+

ALCF Polaris · Image: Argonne National Laboratory

One Aurora node



Multiply by 10,624. The performance is in the blue boxes. The difficulty is in the lines between them.

And a testbed for what comes after the GPU



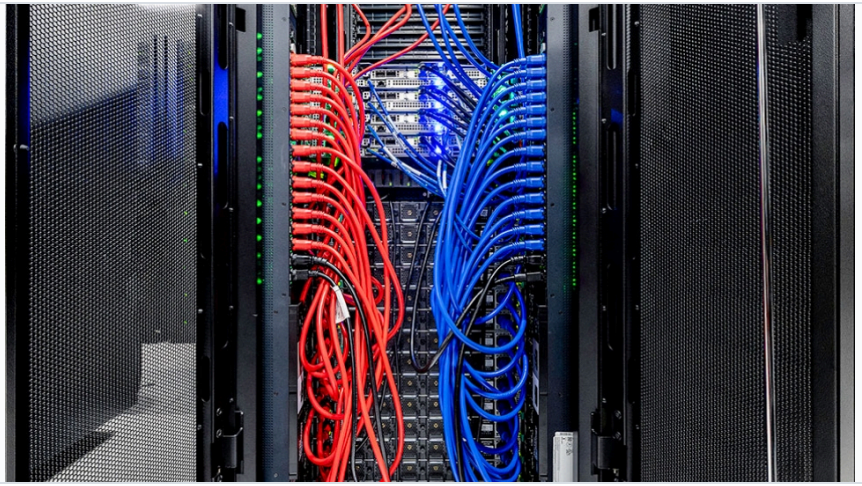
Cerebras CS-3, 4 nodes, 900,000 cores per accelerator, 125 PFLOPS FP16 each



Metis (SambaNova SN40L), 32 accelerators; powers the public inference endpoints



GroqRack, 72 accelerators, >750 TFLOPS INT8 each, low-latency inference



Graphcore Bow Pod64, 64 IPUs, 1,472 processing units each

> The **ALCF AI Testbed**, evaluating AI accelerators that are not GPUs

AI Testbed systems, side by side

System	Size	Per accelerator	Peak (FP16)
Cerebras CS-3	4 nodes · 1 wafer each	900,000 cores	**125,000 TFLOP/s**
SambaNova SN40L (Metis)	32 accel · 16×2	1,040 units	**637.5 TFLOP/s**
GroqRack	72 accel · 9×8	5,120 vector ALUs	**>188 TFLOP/s**
Graphcore Bow Pod64	64 accel · 4×16	1,472 units	**>250 TFLOP/s**
Tenstorrent Galaxy (soon)	2 servers	80 Tensix cores/proc	**292 TFLOP/s**

Five accelerators from five vendors, each a different bet on the same problem

Peak figures are vendor best-case, typically assuming sparsity

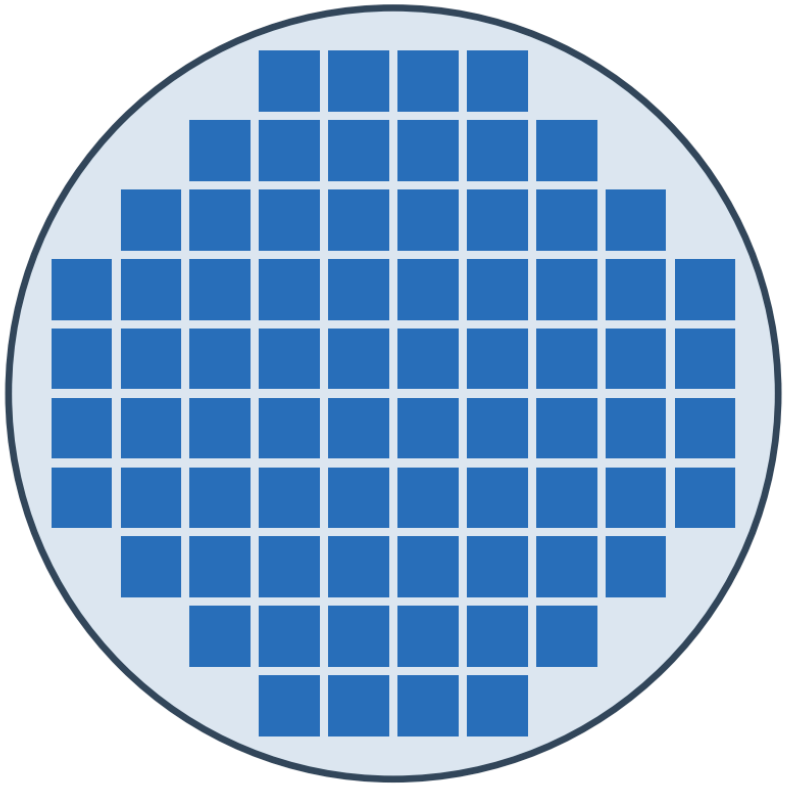
alcf.anl.gov/alcf-ai-testbed *(retrieved Sep 2026)*

05

Wafer-Scale

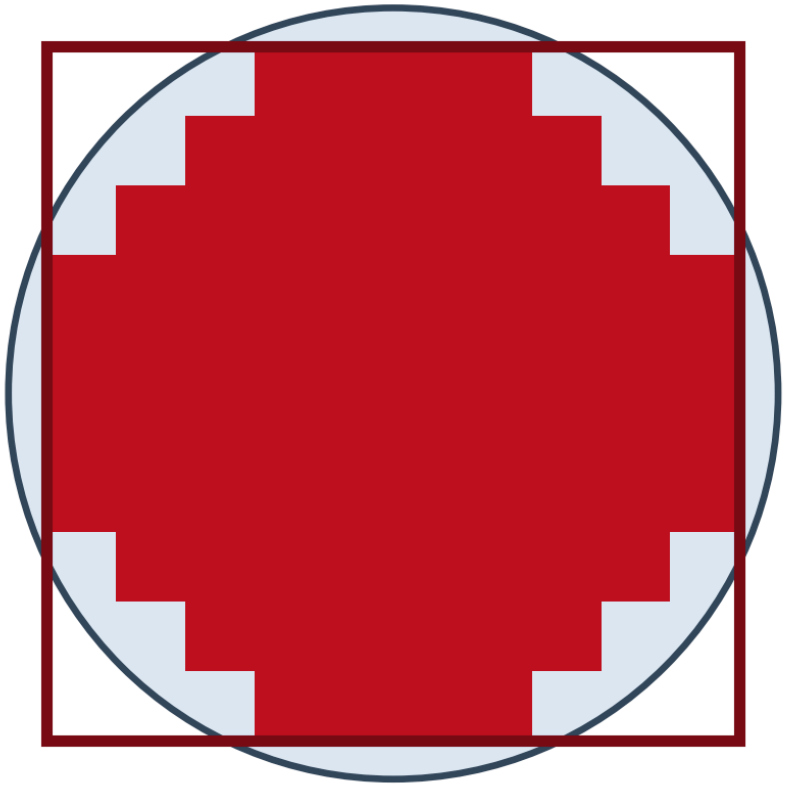
Wafer-scale integration

Normal chips: the wafer is cut into hundreds of dies



each die becomes one CPU or GPU

Wafer-scale: the wafer is left whole

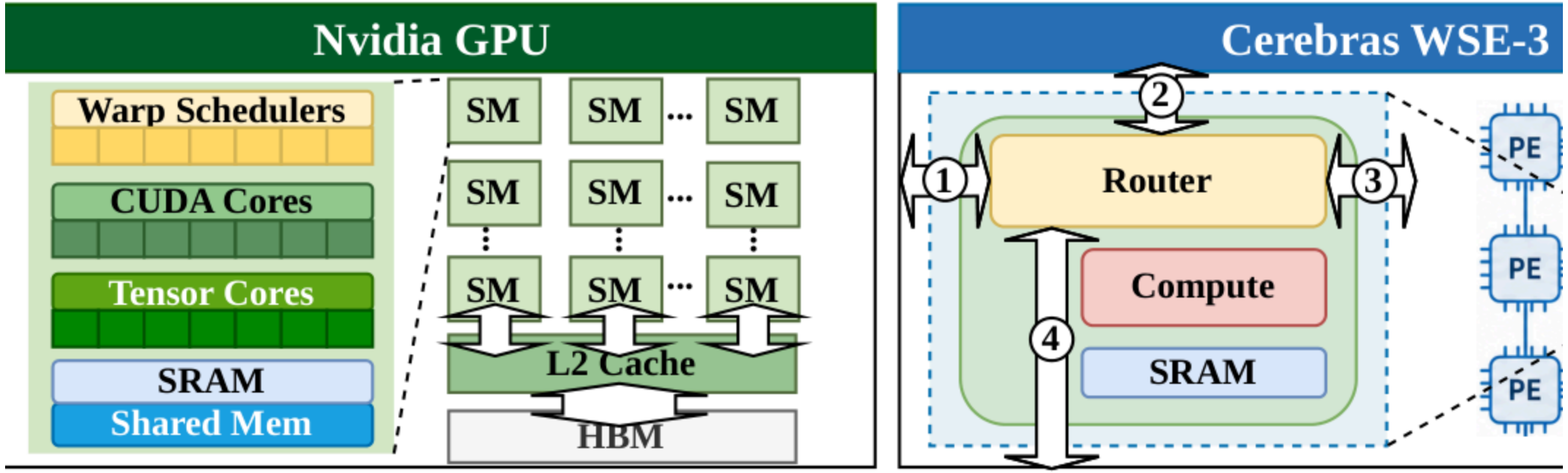


one chip, 900,000 cores

Conventional fabrication cuts a silicon wafer into hundreds of dies

Cerebras keeps the wafer whole as a single chip of 900,000 cores

Compute and memory are fused per tile



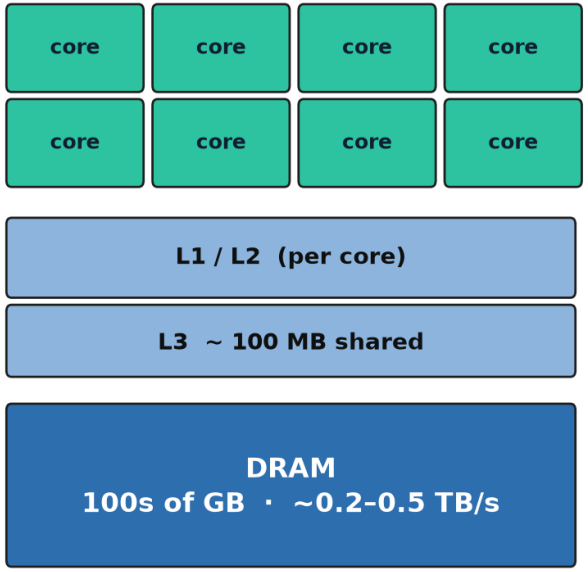
GPU: deep hierarchy — cores, cache, then HBM off the compute die

WSE: each processing element holds compute, its own SRAM, and a router together

Where the memory sits

CPU
few fat cores, deep cache

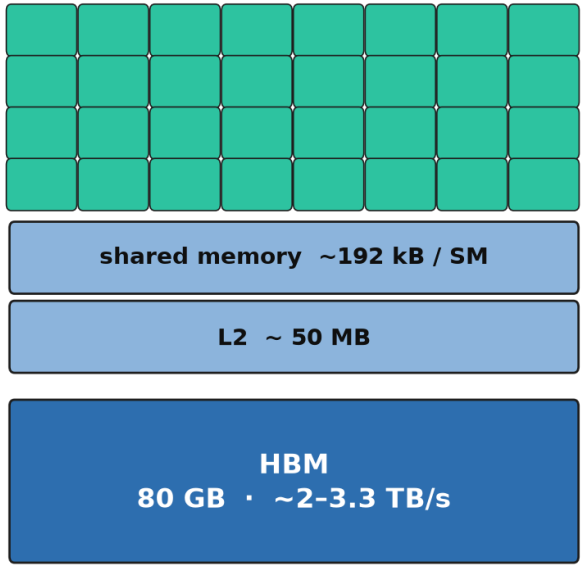
8–128 cores · a few threads each



latency hidden by caches + out-of-order execution

GPU
many cores, shallower hierarchy

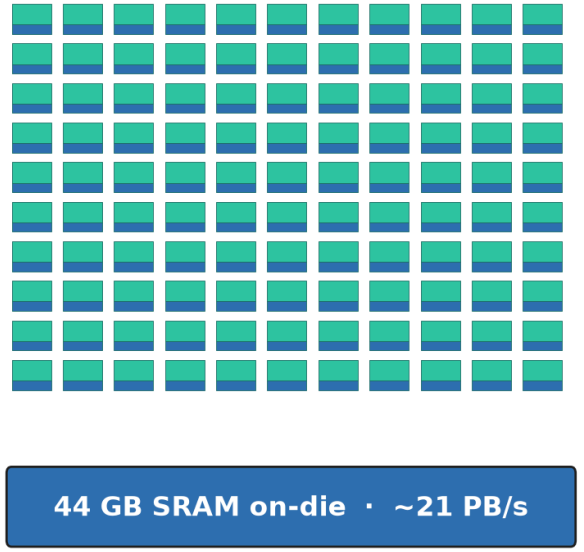
~108 SMs · thousands of threads



latency hidden by oversubscription (swap warps on stall)

Wafer-Scale Engine
no hierarchy at all

900,000 PEs · 48 kB SRAM each



no cache, no HBM: compute and memory fused in every tile

cores / compute · on-chip cache / SRAM · off-chip memory

Green is compute, light blue is on-chip cache, dark blue is off-chip memory

The hierarchy grows shallower left to right; on the wafer, memory fuses into each tile

Where a wafer-scale engine fits

Well suited

- Low arithmetic per byte: stencils, sparse solvers, particle transport
- Iterative — the same data is reused for many steps
- Working state fits within on-chip memory

Poorly suited

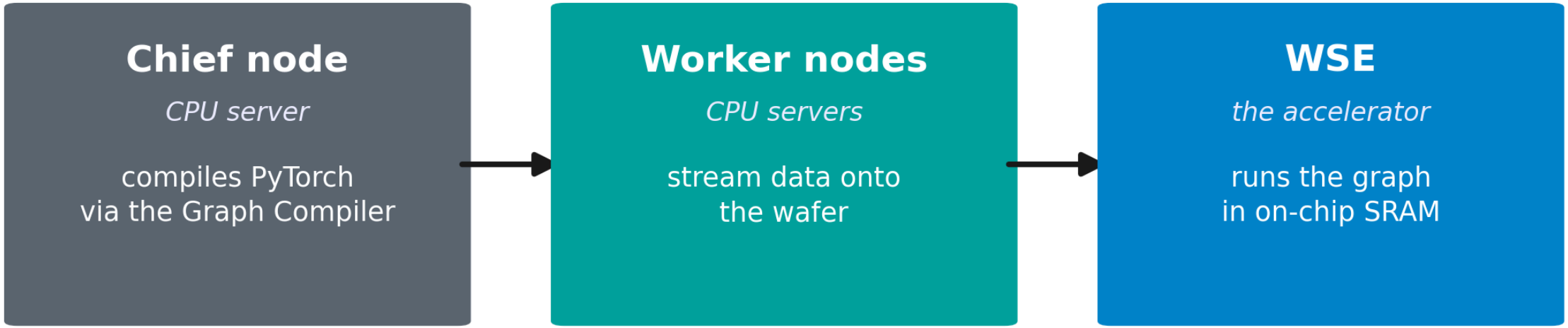
- One-shot work dominated by loading the data
- Working set larger than on-chip memory
- Throughput across many independent users

The advantage comes from keeping the working set resident and reusing it.

Suited to iterative, low-intensity work whose state fits on-chip

Poorly suited to one-shot work, or working sets larger than on-chip memory

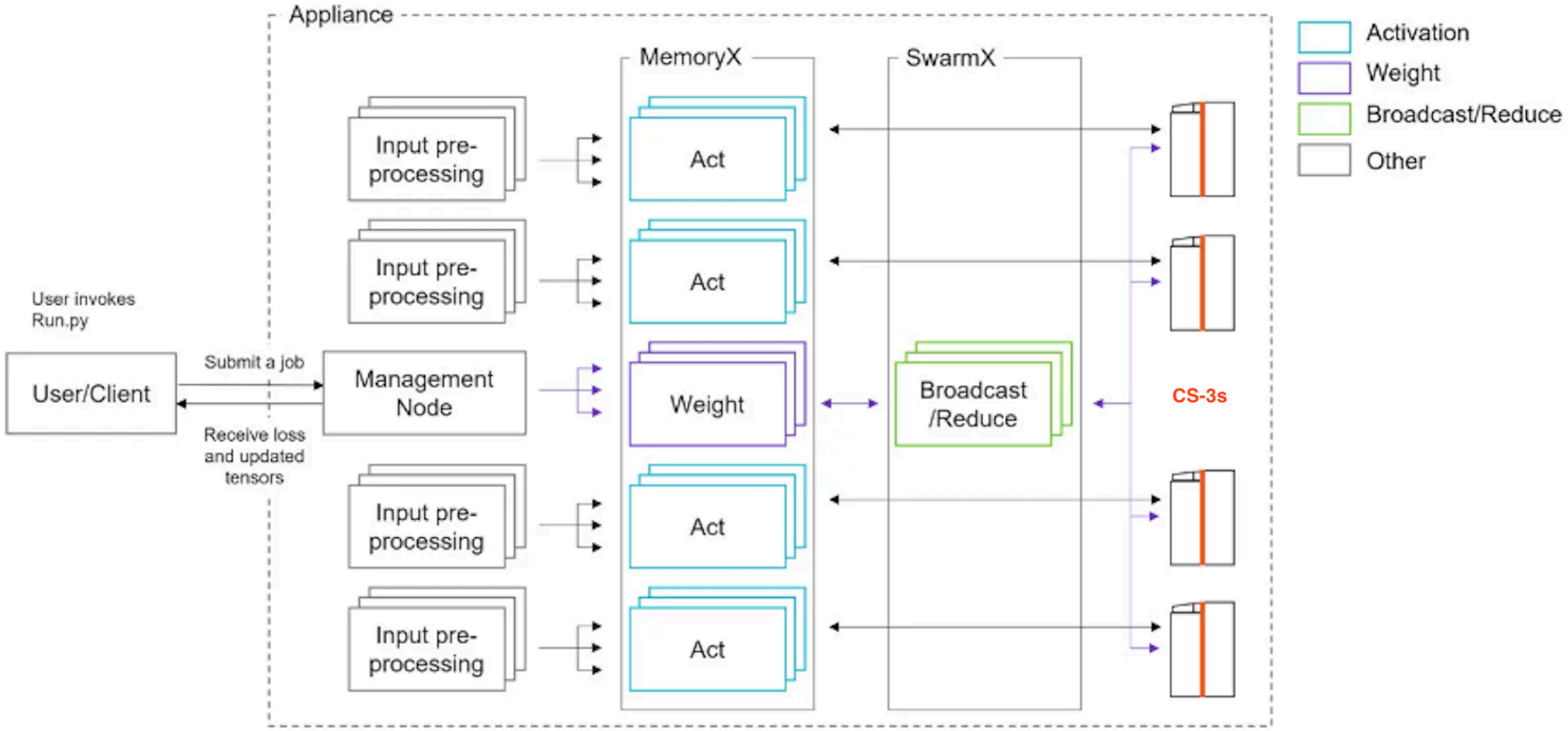
How a CS-3 is organized



Write or port a model → compile → workers stream data → the wafer executes.

> Chief node compiles the model; worker nodes stream data; the wafer executes
>
> You write PyTorch; the Cerebras Graph Compiler places it on the wafer

The Cerebras CS-3 cluster at Argonne



CS-3 wafer: 900K cores · 48 KB SRAM each · 44 GB on-chip · programmed in PyTorch

ALCF cluster: 4 CS-3 WSEs + 4 worker nodes, 4 activation servers, 24 MemoryX (2×12), 8 SwarmX — supports models up to ~200 B parameters

Run as an appliance: submit a job; it handles data streaming, IO, and device orchestration (Weight Streaming for pre-training / fine-tuning)

Two execution modes

Pipeline mode

models below ~1 billion parameters

entire model resident on-chip

data streams through

Weight-streaming mode

models larger than on-chip memory

training data resident

weights stream in layer by layer

Pipeline: the whole model is resident on-chip (models below ~1B parameters)

Weight-streaming: data stays on-chip, weights stream in layer by layer

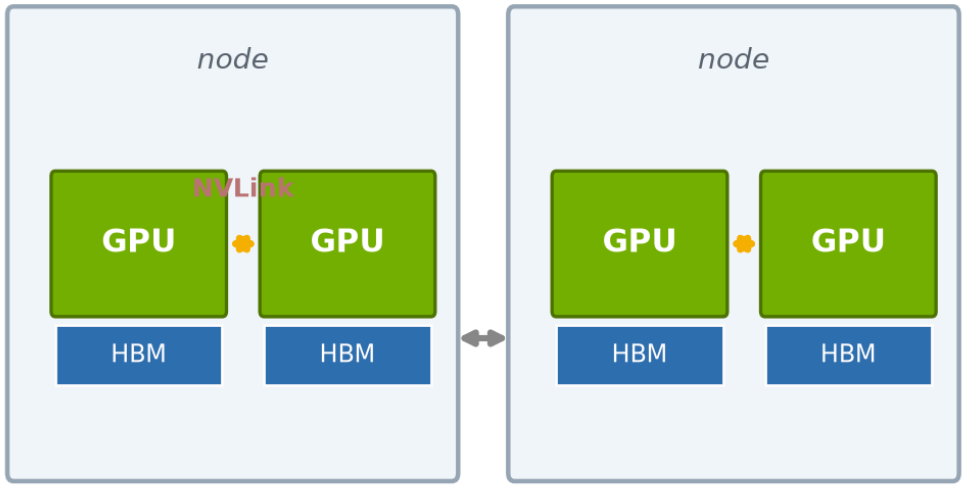
Cerebras WSE-3 and NVIDIA GB300

	Cerebras WSE-3	NVIDIA GB300
Memory	44 GB on-chip SRAM	288 GB off-chip HBM3
Bandwidth	21 PB/s	8 TB/s
Interconnect	on-wafer silicon fabric	NVLink between chips
Execution model	MIMD (independent cores)	SIMT (threads)
Optimized for	single job, latency	many users, throughput
Scaling	whole graph on one wafer	across many chips

> The wafer holds the whole graph in one piece of silicon
> The GPU scales across many chips through a memory hierarchy

Two ways to get bigger

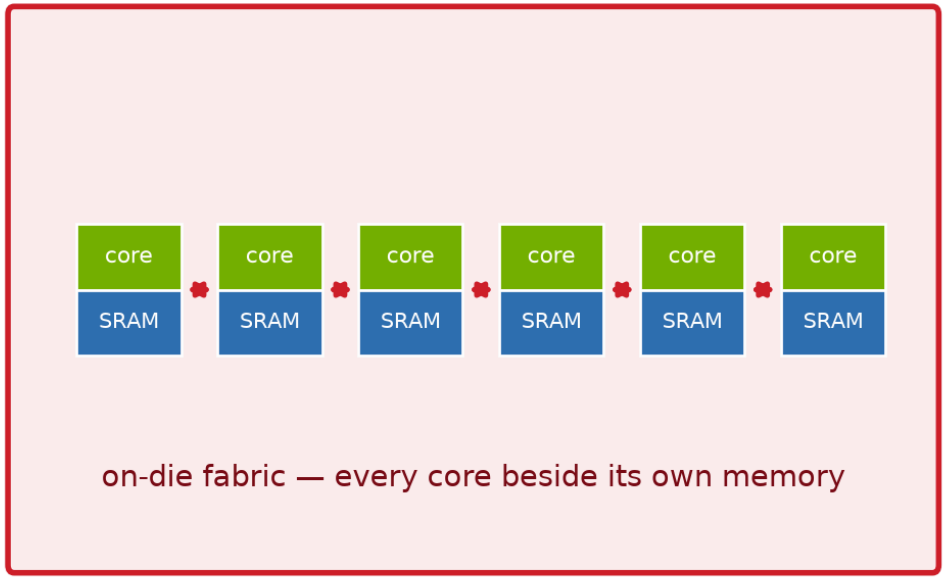
NVIDIA: GPUs + HBM, joined by NVLink



network between nodes

you split the model by hand across GPUs

Cerebras: cores + SRAM, on one wafer



MemoryX / SwarmX join wafers; scaling stays data-parallel

GPU cluster: scale across chips with hand-written data/tensor/pipeline parallelism

Wafer-scale: one chip already; add wafers with data-parallel, near-linear scaling

Where each one wins

Cerebras wins

Real-time chat & agents
lowest time-to-first-token: on-chip SRAM latency beats HBM

Single-stream decode
each token waits on the last; thousands of tokens/s per user

Dense scientific simulation
stencils and dataflow kernels on the on-die fabric

NVIDIA GPUs win

Massive batched throughput
thousands of concurrent requests via HBM memory pooling

Training & general purpose
the mature, ubiquitous CUDA software ecosystem

Flexible deployment
broad pipelines; low onboarding friction

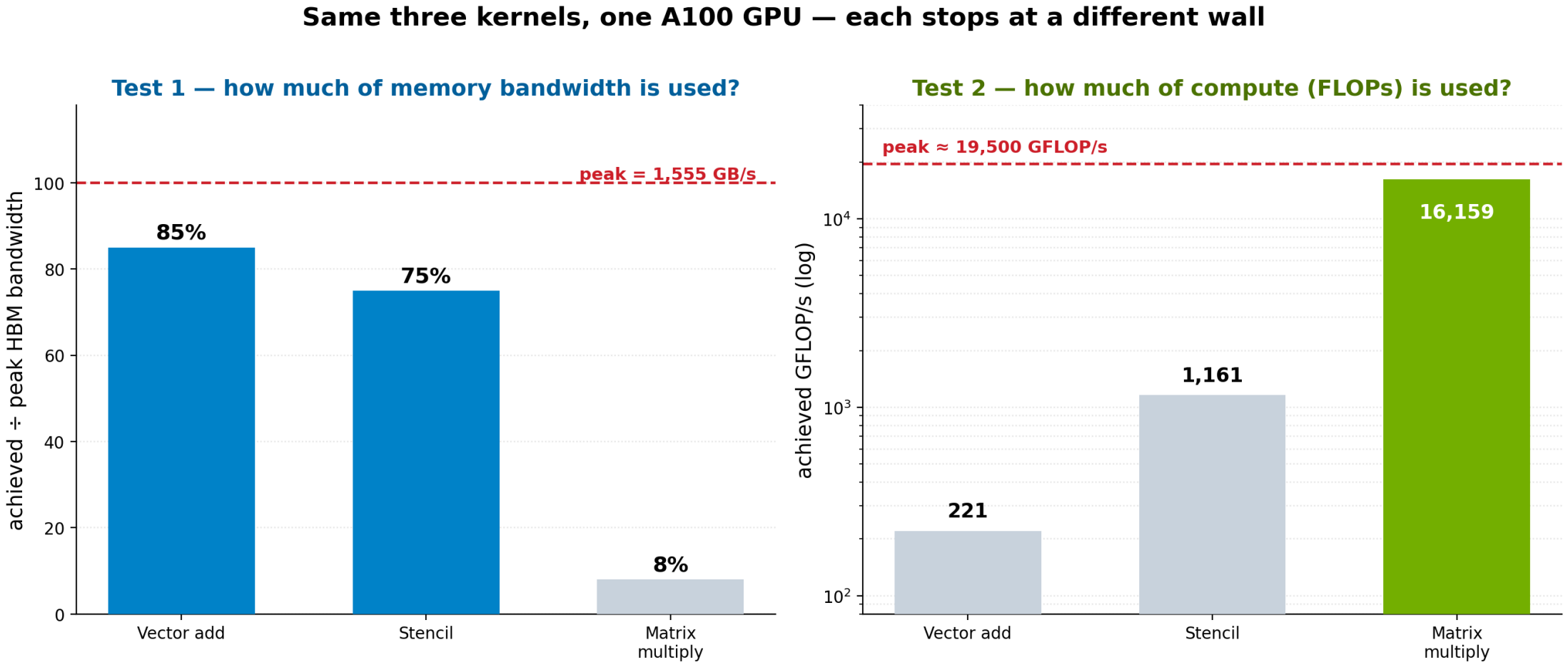
Cerebras: latency-bound, single-stream, on-chip work. NVIDIA: throughput, batching, ecosystem breadth.

The choice follows the workload, not a single "faster" number

06

Findings

Each kernel stops at a different wall

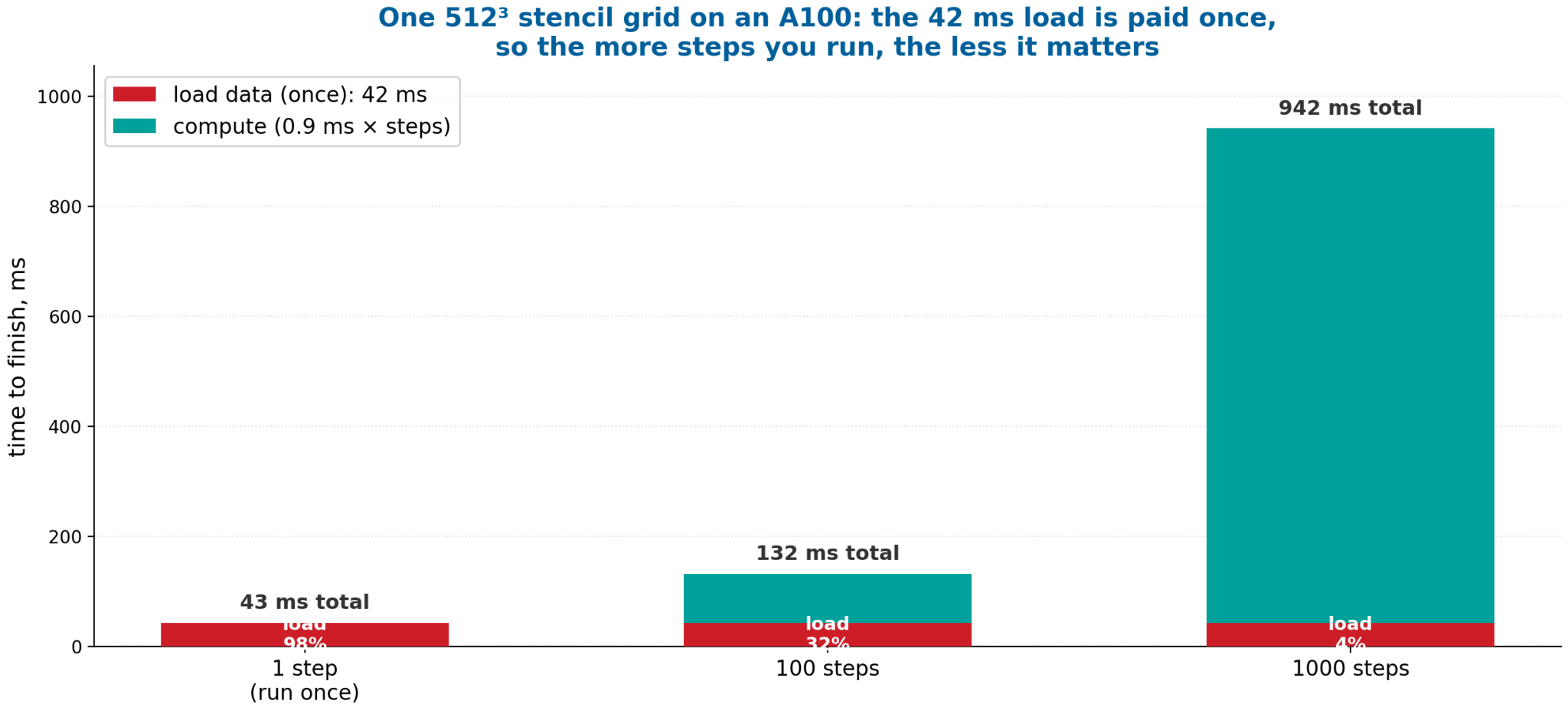


Vector add & stencil hit the memory wall (left); matrix multiply hits the compute wall (right).

We ran the same three kernels on one A100 and measured how close each gets to its two ceilings: memory bandwidth (left) and peak FLOPs (right)

Vector add and stencil top out at the memory wall (75–85% of bandwidth); matrix multiply tops out at the compute wall (16 of ~19.5 TFLOP/s) — the hardware isn't the limit, the kernel is

Loading data is a one-time cost — reuse pays it off



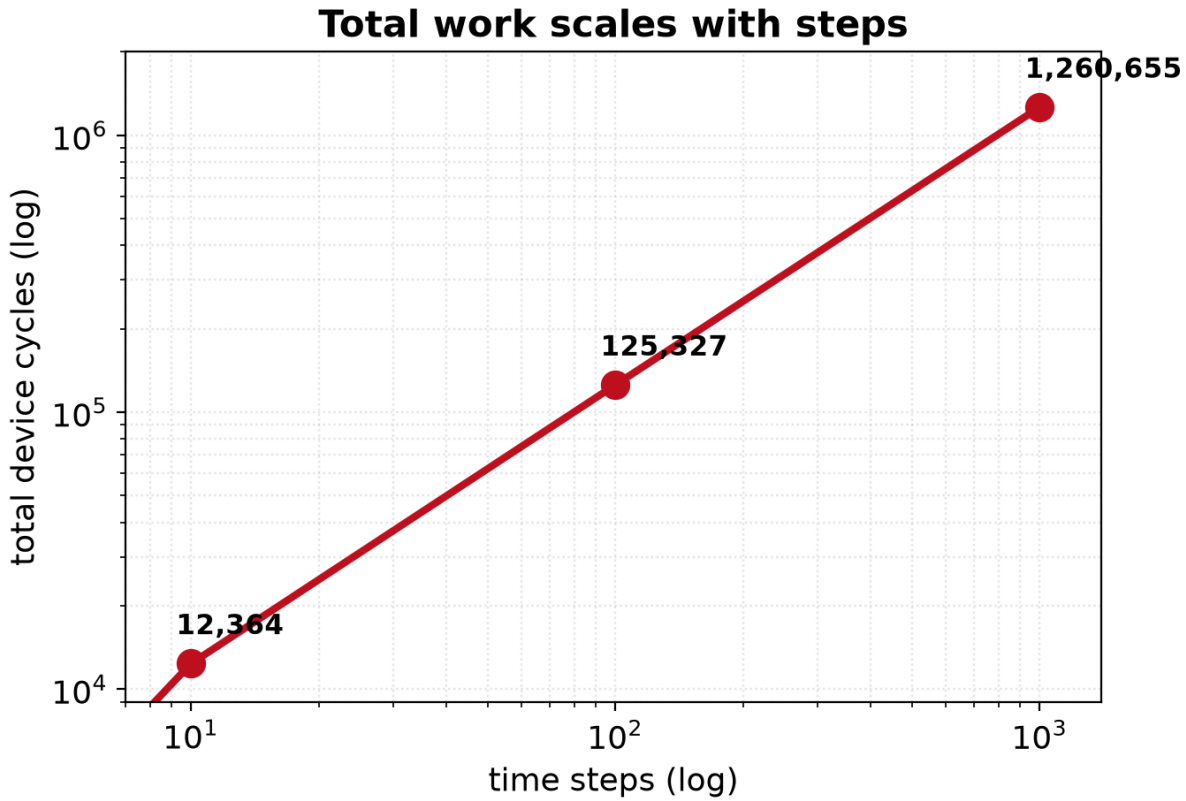
Loading is 98% of the time for a single step, but only 4% once the data is reused for 1000 steps.

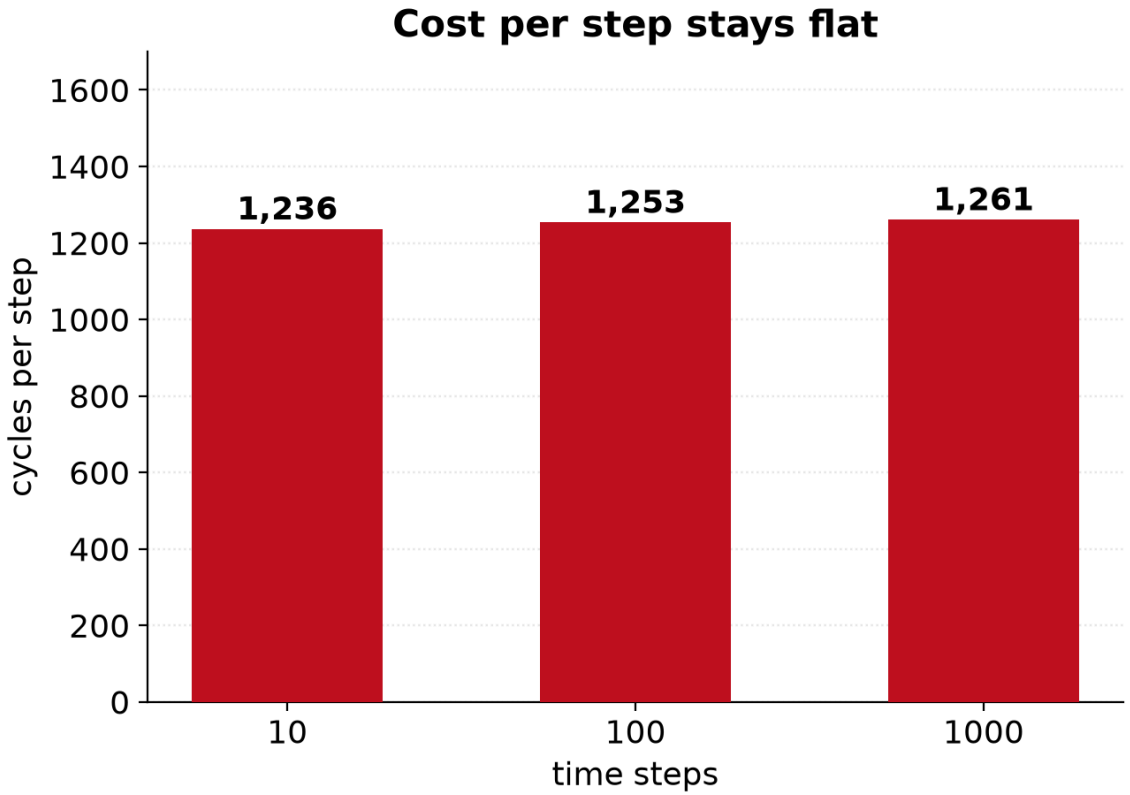
We put one 512³ grid (a cube of 512×512×512 cells) on the A100 once (42 ms), then ran the stencil for 1, 100, and 1000 steps — a "step" is one update of the whole grid (0.9 ms each)

Load is 98% of the time for a single step but only 4% after 1000 steps — reuse hides the load, which is exactly what keeping data resident on the wafer does

On the wafer, per-step cost stays flat

Wafer stencil: the field stays resident between steps





25-pt stencil, 10x10x10, WSE-2 simulator. ~1,250 cycles/step across 10-1000 steps = ~1.47 us at 850 MHz. Measured 2026-09-19.

A 25-point stencil (each cell reads 25 neighbours) on a 10×10×10 grid, run for 10 / 100 / 1000 steps

~1,250 cycles per step no matter how many steps, because the grid stays resident on-chip — no reloading between steps

Reported wafer-scale results in the literature

Seismic imaging

TotalEnergies · Jacquelin et al.

228×

vs A100

Monte Carlo transport

Argonne · Tramm et al.

130×

vs tuned GPU

Both share the wafer's profile: low intensity, iterative, on-chip state.

Reported by the authors on different problems — not a controlled comparison.

Jacquelin, Araya-Polo, Meng, *Massively scalable stencil algorithm*, arXiv:2204.03775 — arxiv.org/abs/2204.03775

Tramm, Allen, Yoshii, Siegel, Wilson, *Efficient algorithms for Monte Carlo particle transport on AI accelerator hardware*, Computer Physics Communications 298 (2024) 109072 — doi.org/10.1016/j.cpc.2023.109072

07

Conclusion

Matching the machine to the program

CPU

- branching logic
- the operating system
- the largest memory

GPU

- high arithmetic per byte
- dense linear algebra
- neural networks

Wafer-scale

- low arithmetic per byte
- iterative reuse
- state fits on-chip

Measured separately, on different problems — not a like-for-like comparison.

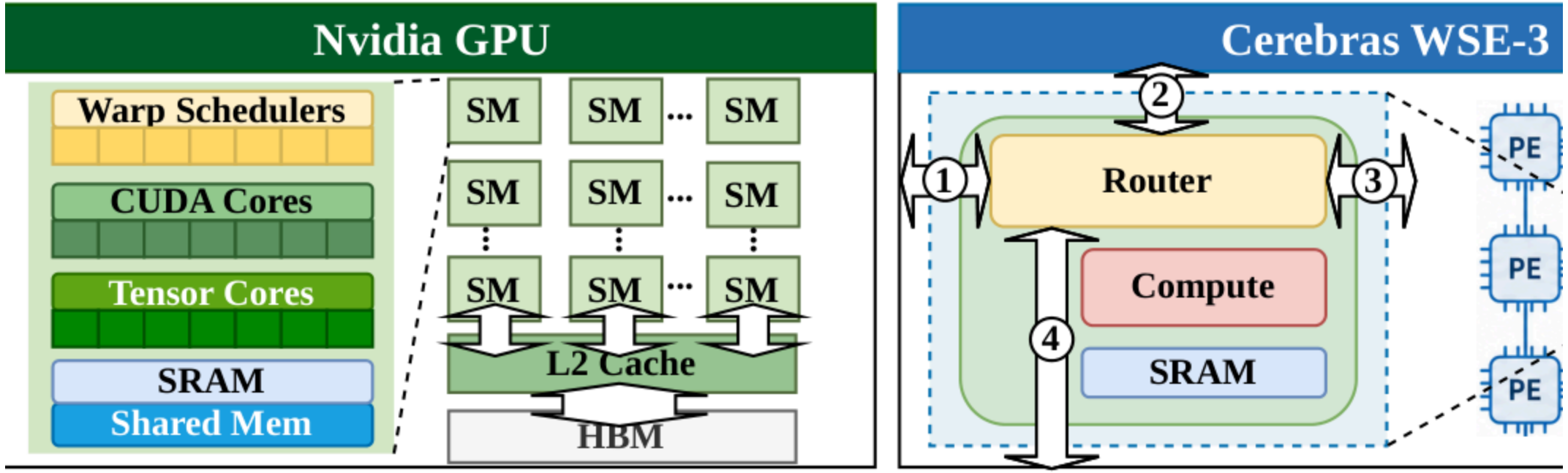
Each machine has a class of program it runs best

Match the workload's arithmetic intensity and working set to the hardware

Summary

- A program is limited by data movement or by arithmetic.
- Most scientific programs are limited by data movement.
- Wafer-scale trades memory capacity for on-chip bandwidth.
- Its advantage is data staying resident across iterations.

Reading list



Luo, Huerta, Vishwanath, Ding, Thakur, Chen, *FABRICA: Agentic CUDA-to-CSL Translation and Optimization for Wafer-Scale Systems*, arXiv:2608.25124 — arxiv.org/abs/2608.25124

Thank you

Ahmedur Rahman Shovon

Postdoctoral Appointee, Argonne National Laboratory

ashovon@anl.gov · arshovon.com

Argonne NATIONAL LABORATORY